

OPAS-DL - Scelte tecnologiche

1. Scopo del Documento

Il presente documento ha lo scopo di descrivere i test effettuati e le valutazioni condotte per individuare la tecnologia più idonea al rinnovo del software desktop OPAS-DL attualmente in uso e a definire gli step successivi per lo sviluppo dell'applicativo.

2. Contesto

Il software attualmente in uso è un'applicazione monolitica sviluppata in Visual Basic e progettata esclusivamente per il sistema operativo Windows. All'interno dello stesso applicativo sono integrati sia i driver di comunicazione con gli strumenti sia l'interfaccia grafica, senza una chiara separazione tra i processi di acquisizione dei dati e quelli di visualizzazione. Questa architettura comporta una significativa complessità nella manutenzione del codice e limita fortemente le possibilità di estensione, evoluzione e riutilizzo dei singoli componenti. Alla luce di tali criticità, si è resa necessaria una valutazione di tecnologie alternative per il rinnovo dell'applicazione, con l'obiettivo di aumentare la flessibilità e la sostenibilità del software nel tempo, mantenendo al contempo le attuali caratteristiche di semplicità d'uso e di installazione.

3. Tecnologie Valutate per la GUI

Nel processo di valutazione sono state prese in considerazione diverse opzioni tecnologiche, rappresentative dei principali approcci allo sviluppo di applicazioni desktop e ibride.

- **Electron.js**, come framework per lo sviluppo di applicazioni desktop multiplatforma basate su tecnologie web, con accesso alle risorse locali del sistema.
- **Applicazioni web eseguite tramite browser**, caratterizzate da un'architettura centralizzata e da una ridotta necessità di installazione lato client, ma con limitazioni nell'accesso alle risorse locali.
- **Tecnologie desktop native** (es. C++ / .NET), valutate per le elevate prestazioni e il ridotto consumo di risorse, a fronte di una maggiore complessità di sviluppo e manutenzione.
- **Soluzioni basate su Python**, tipicamente realizzate mediante framework GUI (es. PyQt, Tkinter) o come servizi locali affiancati da un'interfaccia grafica, considerate per la rapidità di prototipazione e l'ampia disponibilità di librerie.

3.1. Criteri di Valutazione

Le tecnologie sono state valutate secondo i seguenti criteri:

- produttività di sviluppo
- manutenibilità del codice
- integrazione con il sistema operativo
- facilità di deploy multiplatforma
- accesso alle risorse locali
- maturità dell'ecosistema
- curva di apprendimento

- sostenibilità nel tempo

3.2 Scenari di Test

E' stato realizzato un prototipo per verificare:

- avvio e ciclo di vita dell'applicazione
- comunicazione tra processi (main / renderer)
- gestione di processi in background
- accesso al file system locale
- packaging e distribuzione dell'applicazione

4. Risultati dei Test

4.1 Electron.js

Vantaggi:

- Utilizzo di tecnologie web standard (React o stack web classico)
- Integrazione efficace con il sistema operativo
- Semplificazione del processo di build e distribuzione multiplatforma

Inoltre, Electron.js è utilizzato con successo in numerose applicazioni desktop mature e ampiamente diffuse, tra cui:

- **Visual Studio Code**
- **Postman**
- **Slack**
- **Microsoft Teams**
- **Discord**
- **Notion**
- **GitHub Desktop**
- **Obsidian**

Questi esempi dimostrano l'idoneità di Electron allo sviluppo di applicazioni desktop complesse, manutenibili e utilizzate in produzione su larga scala.

Svantaggi:

- Consumo di memoria superiore rispetto a soluzioni native
- Dimensione del pacchetto di installazione maggiore

4.2 Web Application

Vantaggi:

- Facile aggiornamento e manutenzione del front-end
- Indipendenza dalla piattaforma dell'utente
- Possibilità di sfruttare stack web standard consolidati

Svantaggi:

- Maggiore complessità per l'integrazione con driver hardware
- Necessità di gestire un server locale o container
- Limitazioni in termini di accesso diretto al sistema operativo

4.3 Desktop Native (C++ / .NET)

Vantaggi:

- Prestazioni ottimali e basso consumo di risorse
- Controllo diretto sul sistema operativo e sull'hardware
- Dimensione del pacchetto di installazione contenuta

Svantaggi:

- Maggiore complessità nello sviluppo e manutenzione
- Necessità di creare versioni separate per ogni piattaforma
- Stack tecnologico meno flessibile per evoluzioni future

4.4 Python

Vantaggi:

- Sviluppo rapido e prototipazione veloce
- Possibilità di creare interfacce grafiche desktop (PyQt, Tkinter)
- Supporto a web server locali per front-end più complessi

Svantaggi:

- Prestazioni inferiori rispetto a soluzioni native
- Distribuzione e gestione delle dipendenze più complessa
- Integrazione con driver hardware a volte richiede librerie specifiche

5. Architettura scelta

Il nuovo software verrà strutturato secondo un'architettura modulare che separa chiaramente l'interfaccia utente dalla gestione dei dati e dei driver degli strumenti.

- **GUI (Electron.js + React):**

L'interfaccia utente è realizzata con Electron.js, sfruttando tecnologie web standard. Questo consente di fornire un'esperienza moderna, interattiva e multiplatforma, mantenendo al contempo un processo di installazione semplice e coerente con l'applicativo originale.

- **Servizio di Lettura Dati (Python):**

La logica di comunicazione con gli strumenti è affidata a un servizio Python dedicato. Questo servizio gestisce l'esecuzione dei drivers in parallelo, garantendo che eventuali malfunzionamenti su uno strumento non interrompano le operazioni sugli altri. L'uso di Python permette anche di inserire facilmente nuovi driver customizzati senza modificare la GUI.

- **Integrazione tra GUI e Servizio Python:**

La comunicazione tra Electron e il servizio Python sarà inizialmente gestita tramite file, come avviene

nell'applicativo attuale. Tuttavia, verrà valutata la possibilità di introdurre meccanismi di comunicazione interprocesso (IPC), che permettano lo scambio diretto di comandi tra la GUI e il servizio Python.

Questa architettura favorirà la manutenibilità, l'estensibilità e la scalabilità del software, permettendo di aggiungere nuove funzionalità o strumenti senza impattare sull'intera applicazione.

6. Prossimi Obiettivi

Nei prossimi mesi sarà definita l'architettura definitiva del software e avviata la fase di sviluppo. La pianificazione prevede di dedicare il **primo trimestre** alla realizzazione del sistema di acquisizione dati dagli strumenti, mentre il **secondo trimestre** sarà focalizzato sullo sviluppo dell'interfaccia utente. Seguendo questo piano, si prevede di effettuare il **deploy del software su alcune stazioni entro luglio 2026**, garantendo un rilascio controllato e progressivo.